

Assignment 4 — Files, Exceptions, and Errors in Python

This project demonstrates file handling and exception management in Python. It covers reading, writing, and appending text files, along with handling potential errors gracefully.

AIM

To learn how to read, write, and append data in files using Python and to handle exceptions such as `FileNotFoundError` and `IOError`.

THEORY

Files in Python are opened using `open(filename, mode)`. Modes include:

'r' = Read, 'w' = Write, 'a' = Append.

Context managers (with `open(...)` as `f`;) ensure safe closure of files.

Exceptions allow handling of runtime errors such as missing files or permission issues.

TASK 1 — Read a File and Handle Errors

This program reads from `sample.txt`, prints its contents, and gracefully handles missing file errors.

```
def read_sample_file(filename='sample.txt'):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            print(f"=== Contents of '{filename}' ===")
            for lineno, line in enumerate(f, start=1):
                print(f"{lineno}: {line.rstrip()}")
            print("=== End of file ===")
    except FileNotFoundError:
        print(f"Error: '{filename}' not found.")
    except PermissionError:
        print(f"Error: Permission denied for '{filename}'.")
    except Exception as e:
        print(f"Unexpected error: {e}")

if __name__ == '__main__':
    read_sample_file()
```

Explanation: The 'try' block attempts to open the file. If it fails, specific exceptions like `FileNotFoundError` or `PermissionError` are caught. The `with` statement ensures the file closes automatically.

TASK 2 — Write, Append, and Read a File

This program writes user input to `output.txt`, appends new data, and then reads and prints the final content.

```
def write_and_append(filename='output.txt'):
    user_input = input("Enter something to write to the file: ").strip()
    try:
        with open(filename, 'w', encoding='utf-8') as f:
            f.write("=== Initial Write ===\n")
            f.write("User entered: " + user_input + "\n")
            f.write("This is the first write operation.\n")

        with open(filename, 'a', encoding='utf-8') as f:
```

```

        f.write("=== Appended Section ===\n")
        f.write("Appending more lines here.\n")
        f.write("Append successful!\n")

    print(f"\n=== Final contents of '{filename}' ===")
    with open(filename, 'r', encoding='utf-8') as f:
        content = f.read()
        print(content.rstrip())
    print("=== End of file ===")

except IOError as e:
    print(f"I/O error: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")

if __name__ == '__main__':
    write_and_append()

```

Explanation: The program demonstrates three file modes ('w', 'a', 'r') for writing, appending, and reading. Errors such as IOError are handled using exception blocks.

SAMPLE OUTPUTS

Example 1 (File exists):

- 1: Python is powerful.
- 2: File handling is simple.

Example 2 (File not found):

Error: 'sample.txt' not found.

Example 3 (Write and append):

```

=== Final contents of 'output.txt' ===
User entered: 25
Append successful!
=== End of file ===

```

LEARNING OUTCOMES

- Learn file operations (read, write, append)
- Handle exceptions using try-except blocks
- Use context managers for safe resource handling
- Understand file modes and error management
- Write robust, maintainable file-based programs